```
For each k ∈ {1,...,49}
    Do 100 runs ...
        Sample a h from p_H
        Build a k-NN model with 4000 training points sampled from p_h(x,y).
        Run k-NN estimate generalisation error (for this run)
          using 1000 test points sampled from p_h(x,y)
    The estimated generalization error (y-axis) is then the mean
      of these 100 ''run'' generalisation errors.
```

Figure 2: Experimental Protocol A: k-NN

```
For each m ∈ {100,500,1000,1500,...,4000}
    Do 100 runs ...
         For each k ∈ {1,...,49}
              Sample a h from p_H
              Build a k-NN model with m training points sampled from p_h(x,y).
              Run k-NN estimate generalisation error (for this run)
                  using 1000 test points sampled from p_h(x,y)
         The estimated optimal k (for this run) is then the k
            with minimal estimated generalisation error.
    The estimated optimal k (y-axis) is then the mean
      of these 100 ''run'' optimal 'k's.
```

Figure 3: Experimental Protocol B: k-NN

2.1.4 Additional Clarifications

Note both the k-NN algorithm and the hypothesis h can produced "undefined" results in certain corner cases for individual $\boldsymbol{x}$'s. Resolve these case by generating either the label or prediction uniformly at random.

3 PART III [30%]

3.1 Questions

Notation: We overload $[\cdot]$ as follows $[n] := \{1, 2, \ldots, n\}$ if n is a positive integer and $[\texttt{pred}] = 1$ if `pred` is a logical predicate which is true and $[\texttt{pred}] = 0$ otherwise.

9. **[10 pts]** *Kernel modification* Consider the function $K_c(\boldsymbol{x}, \mathbf{z}) := c + \sum_{i=1}^{n} x_i z_i$ where $\boldsymbol{x}$, $\mathbf{z} \in \Re^n$.

   (a) For what values of $c \in \Re$ is K_c a positive semidefinite kernel? Give an argument supporting your claim. ("The closer your argument is to a proof the more likely it is to receive full credit.")

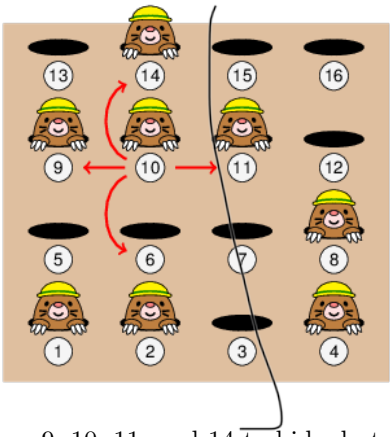
   (b) Suppose we use K_c as a kernel function with linear regression (least squares). Explain how c influences the solution.

10. **[15 pts]** Suppose we perform linear regression with a Gaussian kernel $K_\beta(\boldsymbol{x}, \mathbf{t}) = \exp(-\beta\|\boldsymbol{x} - \mathbf{t}\|^2)$ to train a classifier on a dataset $(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_m, y_m) \in \Re^n \times \{-1, 1\}$. Thus obtaining a function $f : \Re^n \to \Re$ which is of the form $f(\mathbf{t}) = \sum_{i=1}^{m} \alpha_i K_\beta(\boldsymbol{x}_i, \mathbf{t})$. The corresponding classifier is then $\text{sign}(f(\mathbf{t}))$. This classifier depends on the parameter β selected for the kernel. In what scenario will chosen β enable the trained linear classifier to simulate a 1-Nearest Neighbor classifier trained on the same dataset?

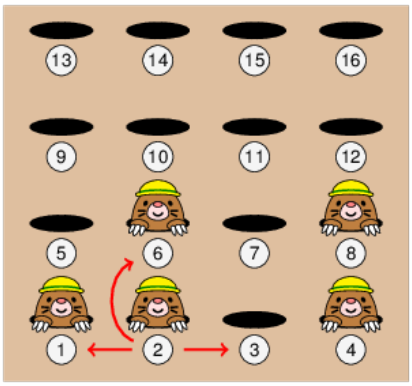
Note $\beta \in \Re$ is a fixed scalar which may be selected as a function of $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_m$ and the test point $\mathbf{t}$, i.e., we may use a function so that $\beta = \hat{\beta}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_m, \mathbf{t})$ an exact function $\hat{\beta}(\cdot)$ need not be given just an argument that one exists.

Give an argument supporting your reasoning. ("The closer your argument is to a proof the more likely it is to receive full credit.")

11. **[5pts]**: *Hint: "Think about vector spaces 'broadly'".*
We consider the problem of generalized Whack-A-Mole! We'll play this game on a board with $n \times n$ holes, from which individual moles pop-up. Using a mallet you can then whack a mole and make it go hide underground. As you might know, however, whacking a mole will often have the effect of making other moles rise and come out from the ground. In this version of the game, every time you hit a hole with the mallet you'll cause the hole and the immediate four adjacent holes to change: any moles currently there will hide, while new moles pop-up from previously empty holes. Imagine, for example, that you find yourself in the following game configuration, and you prepare yourself to hit the hole at position 10 with the mallet.

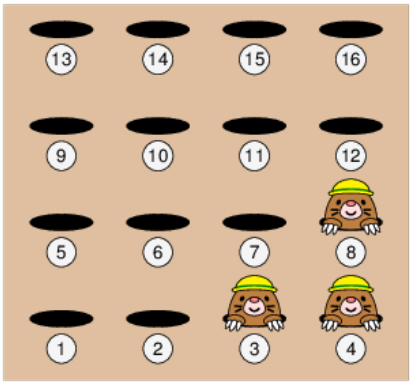


You will cause the moles in positons 9, 10, 11, and 14 to hide, but you'll also make a mole appear at position 6. Thus ending up in the next configuration.



If you then hit the hole at position 2, then all of the moles at positions 1, 2, and 6 will hide; but a new one in position 3 will appear.

Example: Consider the 4×4 board with moles at positions 1,2,4,8,9,10,11, and 14. This is the same configuration of moles as given in the initial illustration above. After hitting holes at positions 10 then 2, you'll end up in the following configuration.



At this point, a single hit to position 4 will clear the board. Thus the hitting sequence $(10, 2, 4)$ is a solution.

Task: Design an algorithm for an $n \times n$ board which, given an initial board configuration, finds a sequence of holes that you can hit in order to empty the board if such a sequence exists. The algorithm must be polynomial in n and you must provide an argument that it is correct. No credit will be given for algorithms that are not polynomial in n. There is no need to implement your algorithm.

9.

a). $K_c(x, z) = c + \sum_{i=1}^{n} x_i z_i \quad , \quad x, z \in \mathbb{R}^n$

Prove PSD

PSD $\Rightarrow$ $a^T K_c a \geq 0 \quad \forall a \in \mathbb{R}^n$

$$a^T K a = \sum_{i,j}^{n} a_i a_j K(i,j)$$

$$= \sum_{i,j}^{n} a_i a_j \langle \phi(x_i), \phi(z_i) \rangle$$

$$= \left\langle \sum_{i=1}^{n} a_i \phi(x_i), \sum_{j=1}^{n} a_j \phi(x_j) \right\rangle$$

$$= \left\| \sum_{i=1}^{n} a_i \phi(x_i) \right\|^2 \geq 0$$

$\Rightarrow$ We if we show $K_c(x, z)$ is an inner product of itself

$a^T K_c a \geq 0 \Rightarrow$ PSD

$$\langle \phi(x), \phi(z) \rangle = \sum_{i=1}^{\hat{}} f(x_i) f(z_i) + C + \sum_{i=1}^{\hat{}} x_i z_i$$

$$\sum x_i z_i = x^T z =$$

$C = f(x_i) f(z_i)$ - constant

$\Rightarrow$

$$\phi(x) = \begin{bmatrix} x_n \\ \sqrt{C} \end{bmatrix}$$

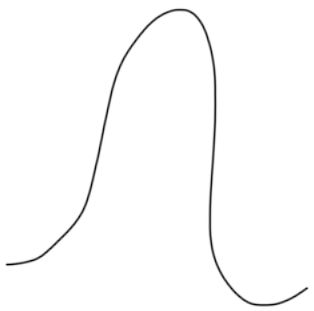
$C > 0$

$C = 0 \Rightarrow \phi(x) = x$

$C < 0$: $K(\phi, \phi) = C < 0$ $\rightsquigarrow$ not PSD

10. $K_\beta(x, t) = \exp\left(-\beta \|x - t\|^2\right)$

$f(t) = \sum_i^l \alpha_i K_\beta(x_i, t)$



As $\beta \to \infty$ $\quad K \to 0$

$$\text{sign}\left(\operatorname*{argmin}_{x_i} \|x_i - t\|^2\right)$$



$f(t)$

$f(t) = \operatorname*{argmin}_{x_i} \|x_i - t\|^2$

$$\hat{y}_{NW}(x) = \frac{1}{k} \sum_{i \in I_x} y_i \qquad k = 1$$

$$\sum_{i \in I_x} y_i$$

$\operatorname{argmin}_{x_i} \|x_i - t\|^2$ ← small

β sufficiently large

$$x^* = \operatorname*{argmin}_{x_i} \|x_i - t\|^2$$

$$\exp(-\beta x^*) \approx 1$$

$$\exp(\beta x_i) \approx 0 \qquad \forall x_i \neq x^*$$

linear regression

$$\beta_1 = \frac{S_{xx}}{S_{xy}} = \frac{\sum_i K(x, t_i)}{K(x, t_i)}$$

$$K(x,t) = \exp(-\beta \|x - t\|^2)$$

$$w = \sum_i \alpha_i K(x_i, t)$$

$$\Rightarrow y(t) = \sum \alpha_i K(x_i, t)$$

$$=$$

$$\alpha = (K^T K)^{-1} K^T y$$

$$y(t) = k(t)^T K^{-1} y \quad , \quad k(t)_i = K(x_i, x)$$

$$\hat{y}(t) = \frac{\sum_i e^{-\beta \|x_i - t\|^2} y_i}{\sum_j e^{-\beta \|x_j - t\|^2}}$$

Let x_* be nearest neighbour

$$\Rightarrow \|x_* - t\|^2 < \|x_i - t\|^2 \quad \forall i \neq *$$

$$\hat{y}(t) = \sum w_i y_i \,, \quad w_i = \frac{e^{-\beta \|x_i - t\|^2}}{\sum_j e^{-\beta \|x_j - t\|^2}}$$

$$\lim_{\beta \to \infty} \frac{w_i}{w_*} = \lim_{\beta \to \infty} \frac{e^{-\beta \|x_i - t\|^2}}{e^{-\beta \|x_* - t\|^2}}$$

$$= \lim_{\beta \to \infty} e^{-\beta (\|x_i - t\|^2 - \|x_* - t\|^2)} = 0$$

$$\Rightarrow \hat{y}(t) = \frac{\frac{\sum e^{-\beta \|x_i - t\|^2} y_i}{e^{-\beta \|x_* - t\|^2}}}{\frac{\sum_j e^{-\beta \|x_j - t\|^2}}{e^{-\beta \|x_* - t\|^2}}} = \frac{y_* + \sum \frac{w_i}{w_*} y_i}{1 + \sum \frac{w_i}{w_*}} \xrightarrow{\beta \to \infty} y_*$$